

Portfolio BTS SIO SLAM - Agrégateur de Veille Technologique

Auteur: Mekaoui Reda

Formation: BTS SIO (SLAM) - Année 2

Date: Décembre 2025

Technologies: Laravel 11, Docker Sail, Blade Templates, RSS/Atom Feeds



SOMMAIRE

1. [Présentation du Projet](#)
 2. [Environnement de Développement](#)
 3. [Architecture Logicielle MVC](#)
 4. [Sécurité & Qualité \(BTS SIO\)](#)
 5. [Le Moteur de Veille Technologique](#)
 6. [Interface Utilisateur](#)
-

1. PRÉSENTATION DU PROJET

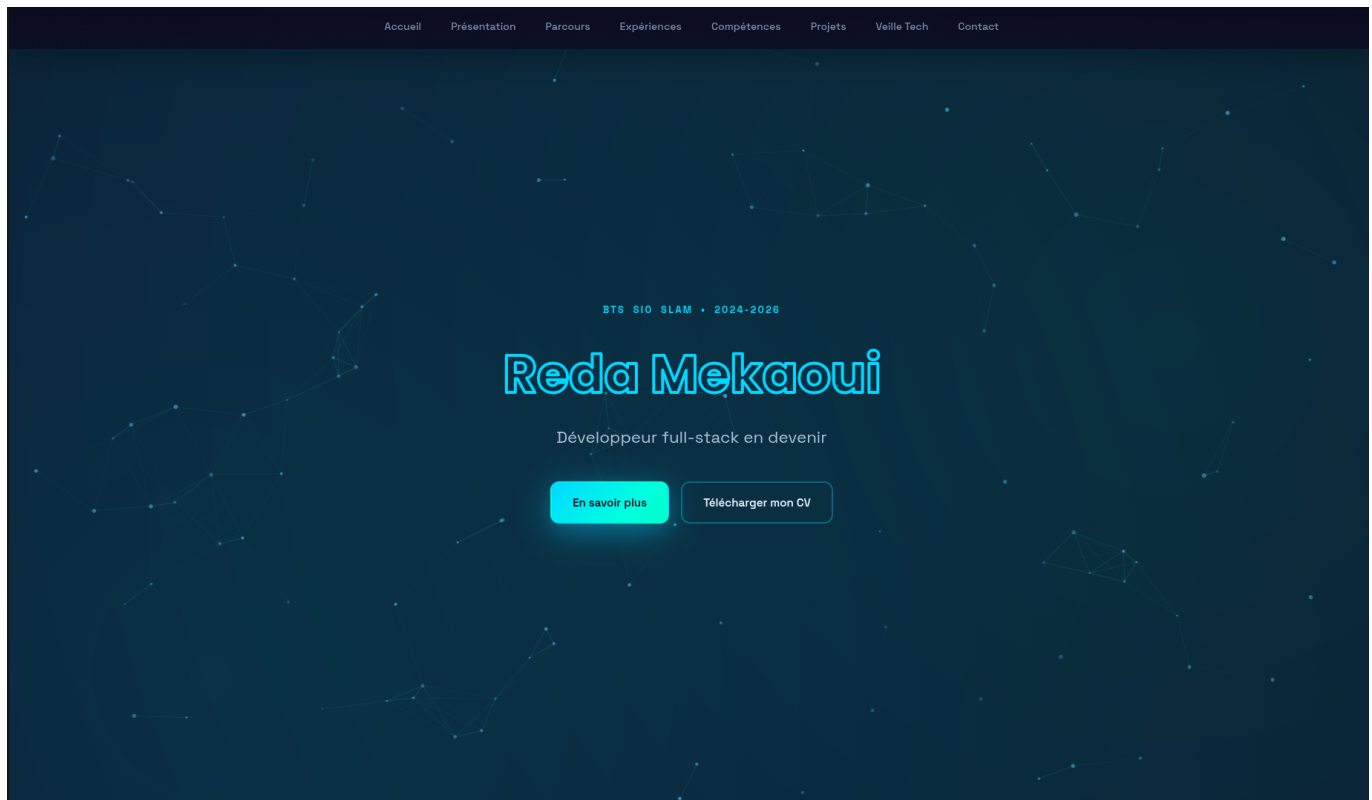
1.1 Contexte et Objectifs

Ce projet a été réalisé dans le cadre de ma formation en **BTS SIO (Services Informatiques aux Organisations)**, option **SLAM (Solutions Logicielles et Applications Métiers)**.

L'objectif n'était pas seulement de créer un CV en ligne, mais de développer une **véritable application web dynamique** capable de démontrer ma maîtrise du développement Backend et Frontend.

Le site répond à trois besoins principaux :

- **Vitrine Professionnelle** : Présenter mon parcours, mes compétences et mes projets de stage (Assuroweb, etc.)
- **Démonstration Technique** : Prouver ma capacité à utiliser un framework moderne (Laravel 11) et une architecture rigoureuse (MVC)
- **Outil de Veille Automatisé** : Intégrer un agrégateur de flux RSS intelligent pour suivre mon sujet de veille en temps réel



- Capture d'écran de la page d'accueil (Hero Section)

- Localisation: En haut de cette section
- Contenu: Le titre "Reda Mekaoui", le sous-titre "Développeur full-stack en devenir", avec les boutons CTA et les particules en arrière-plan
- Dimensions recommandées: 1920x1080px ou full-width
- Légende: *Page d'accueil du portfolio avec design dark theme et animations de particules*

1.2 Le Sujet de Veille Technologique

Une partie centrale de ce portfolio est dédiée à ma veille technologique personnelle.

Thème: "L'IA générative dans les interactions UI complexes"

Problématique: Comment l'Intelligence Artificielle (comme GitHub Copilot ou V0.dev) transforme-t-elle la manière dont nous concevons et codons les interfaces utilisateurs (Frontend) ?

Solution mise en œuvre: J'ai développé un algorithme capable de récupérer des articles depuis Google Alerts, de les filtrer pour ne garder que ceux parlant à la fois d'IA et d'Interface, et de les afficher dynamiquement sur le site.



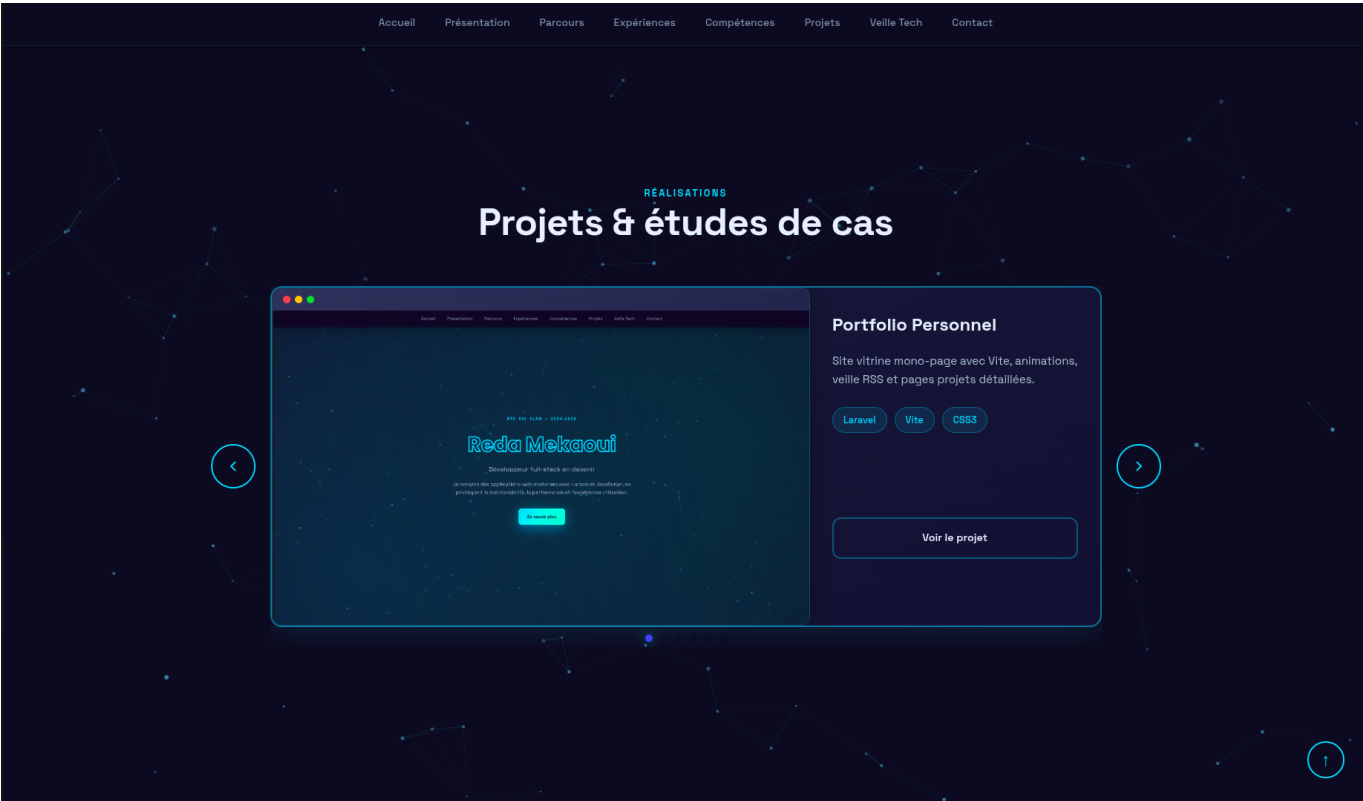
- Section "Veille Technologique"

- Localisation: Après la description du sujet, en bas de la section 1.2
- Contenu: La grille des 6 articles de veille avec leurs badges de catégorie (Frontend AI, AI Context, etc.)
- Dimensions recommandées: 1400x600px
- Légende: *Agrégateur de veille avec 6 articles filtrés automatiquement selon le sujet IA + Interactions UI*

1.3 Fonctionnalités Clés

Le projet intègre plusieurs fonctionnalités avancées :

- **Carrousel de Projets** : Navigation interactive (Swiper.js) pour explorer mes 6 réalisations
- **Moteur de Recherche RSS** : Parsing et tri automatique de flux XML (Atom/RSS) avec filtrage intelligent
- **Formulaire de Contact AJAX** : Envoi d'emails sans rechargement de page avec validation en temps réel
- **Mode Sombre (Dark Mode)** : Design ergonomique avec thème néon cyan/violet adapté aux développeurs
- **Animations Fluides** : SVG animé (Anime.js), carrousel tactile (Swiper), particules Canvas



- Carrousel de projets
 - Localisation: Section des fonctionnalités, après la liste à puces
 - Contenu: Capture du carrousel avec 2-3 cartes projets visibles (Portfolio, Assuroweb, Next2You)
 - Dimensions recommandées: 1400x400px
 - Légende: *Carrousel interactif affichant 6 projets avec mockups, descriptions et stacks technologiques*

2. ENVIRONNEMENT DE DÉVELOPPEMENT

2.1 Architecture Docker avec Laravel Sail

Le projet repose sur une **conteneurisation complète via Laravel Sail** (surcouche Docker officielle de Laravel), garantissant que l'environnement de développement est **identique à celui de production**, sans polluer la machine hôte.

Avantages de Laravel Sail:

-  Aucune installation de PHP, Composer, Node.js sur la machine locale
-  Isolation totale des dépendances (versions PHP, extensions)
-  Configuration reproductible (commit du `docker-compose.yml`)
-  Démarrage facile avec `./vendor/bin/sail up -d`

2.2 Stack Technique Complète

Framework:	Laravel 11.x (PHP 8.2)
Serveur Web:	Nginx (Docker)
Base de données:	MySQL 8 (Docker)
Frontend:	Blade Templates + JavaScript Vanilla (ES6)
Bundler Assets:	Vite (compilation CSS/JS)

```
Package Manager: Composer (PHP), NPM (Node.js)
Conteneurisation: Docker + Docker Compose
```

2.3 Installation Étape par Étape

Étape 1 : Cloner le projet et installer les dépendances PHP

```
# Cloner depuis GitHub (ou récupérer les fichiers)
git clone https://github.com/votre-repo/portfolio.git
cd portfolio

# Installer Composer et les dépendances PHP via un conteneur temporaire
# (Permet d'installer sans avoir PHP localement)
docker run --rm \
  -u "$(id -u):$(id -g)" \
  -v "$(pwd):/var/www/html" \
  -w /var/www/html \
  laravelsail/php82-composer:latest \
  composer install --ignore-platform-reqs
```

Explication:

- `-u "$(id -u):$(id -g)"` : Préserve les permissions de fichiers
- `-v "$(pwd):/var/www/html"` : Monte le répertoire local dans le conteneur
- `laravelsail/php82-composer:latest` : Image Docker officielle avec PHP 8.2 + Composer

Étape 2 : Créer le fichier d'environnement

```
# Copier le fichier exemple en .env (fichier de config local)
cp .env.example .env

# Générer une clé d'application unique (sécurité Laravel)
./vendor/bin/sail artisan key:generate
```

Étape 3 : Lancer l'environnement Docker

```
# Démarrer tous les services (Nginx, MySQL, Redis, etc.)
# -d = detached mode (fonctionne en arrière-plan)
./vendor/bin/sail up -d

# Vérifier que les services sont lancés
./vendor/bin/sail ps
```

Résultat attendu:

NAME	STATUS	PORTS
portfolio_laravel	Up	0.0.0.0:80->80/tcp
portfolio_mysql	Up	3306/tcp
portfolio_redis	Up	6379/tcp

Étape 4 : Installer les dépendances Frontend (NPM)

```
# Installer les packages JavaScript (Swiper, Anime.js, Vite, etc.)
./vendor/bin/sail npm install

# Compiler les assets en mode développement (watch)
./vendor/bin/sail npm run dev
```

Cela va :

- Télécharger **swiper**, **animejs** (définis dans **package.json**)
- Compiler **resources/css/style.css** et **resources/js/script.js** avec Vite
- Générer les fichiers dans **public/hot** ou **public/build**

Étape 5 : Migration de la base de données (optionnel)

```
# Créer les tables (users, cache, jobs, etc.)
./vendor/bin/sail artisan migrate

# Remplir avec des données de test (seeders)
./vendor/bin/sail artisan db:seed
```

Étape 6 : Accéder à l'application

Ouvrir le navigateur à **http://localhost**

2.4 Commandes Courantes

```
# Démarrer/Arrêter les services
./vendor/bin/sail up -d      # Lancer en arrière-plan
./vendor/bin/sail down      # Arrêter tous les conteneurs

# Exécuter des commandes Laravel
./vendor/bin/sail artisan list # Voir toutes les commandes disponibles
./vendor/bin/sail artisan tinker # Interpréteur PHP interactif

# Exécuter du PHP/Composer
./vendor/bin/sail php -v      # Vérifier la version PHP
./vendor/bin/sail composer require package-name
```

```
# Exécuter du Node.js/NPM
./vendor/bin/sail npm run build # Compiler les assets en production
./vendor/bin/sail npm run dev   # Mode watch (développement)

# Accéder à MySQL
./vendor/bin/sail mysql        # Client MySQL interactif
```

2.5 Variables d'Environnement Importantes

Fichier `.env` à configurer :

```
APP_NAME="Portfolio"
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost

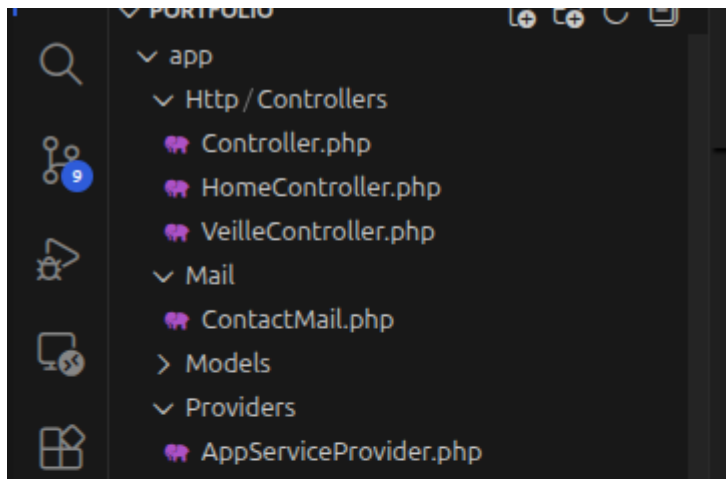
DB_CONNECTION=mysql
DB_HOST=mysql
DB_PORT=3306
DB_DATABASE=portfolio
DB_USERNAME=sail
DB_PASSWORD=password

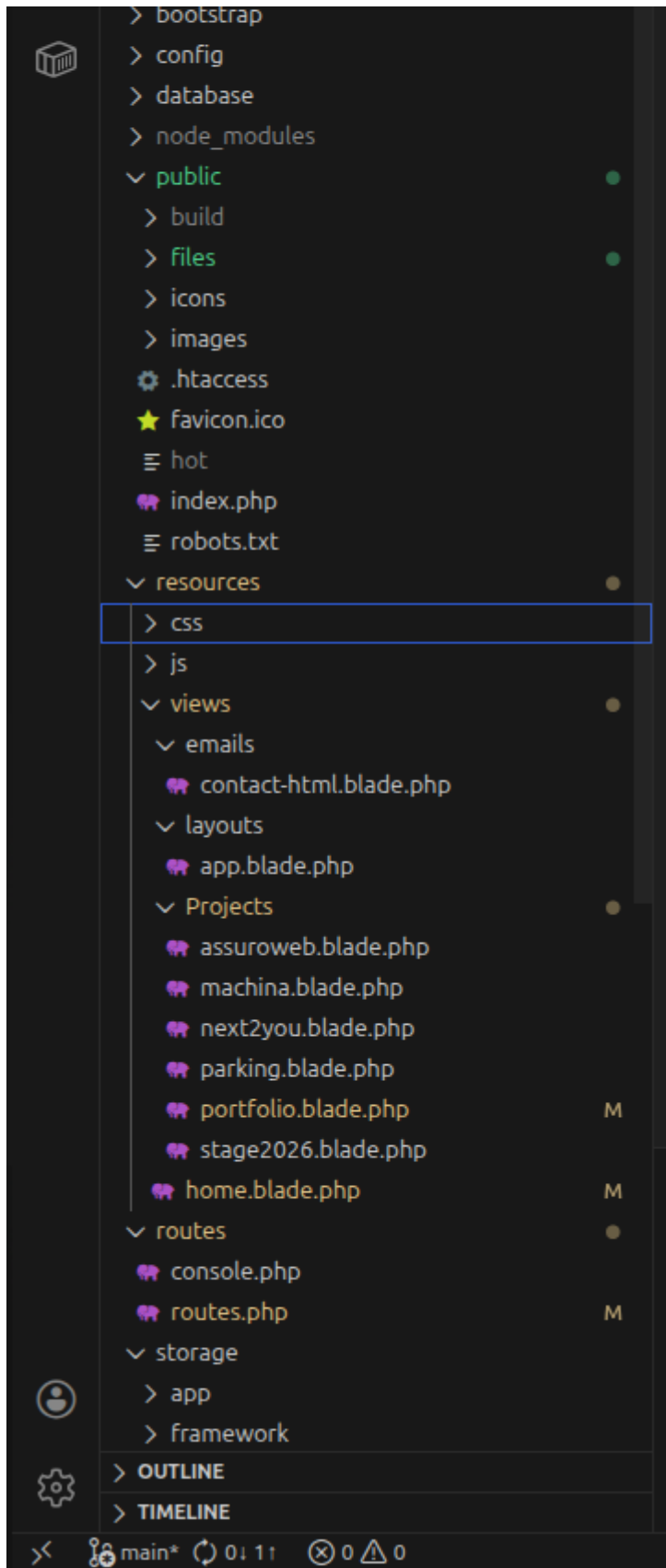
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=votre-email@gmail.com
MAIL_PASSWORD=votre-app-password
MAIL_FROM_ADDRESS=votre-email@gmail.com
```

3. ARCHITECTURE LOGICIELLE MVC

3.1 Pattern MVC dans Laravel

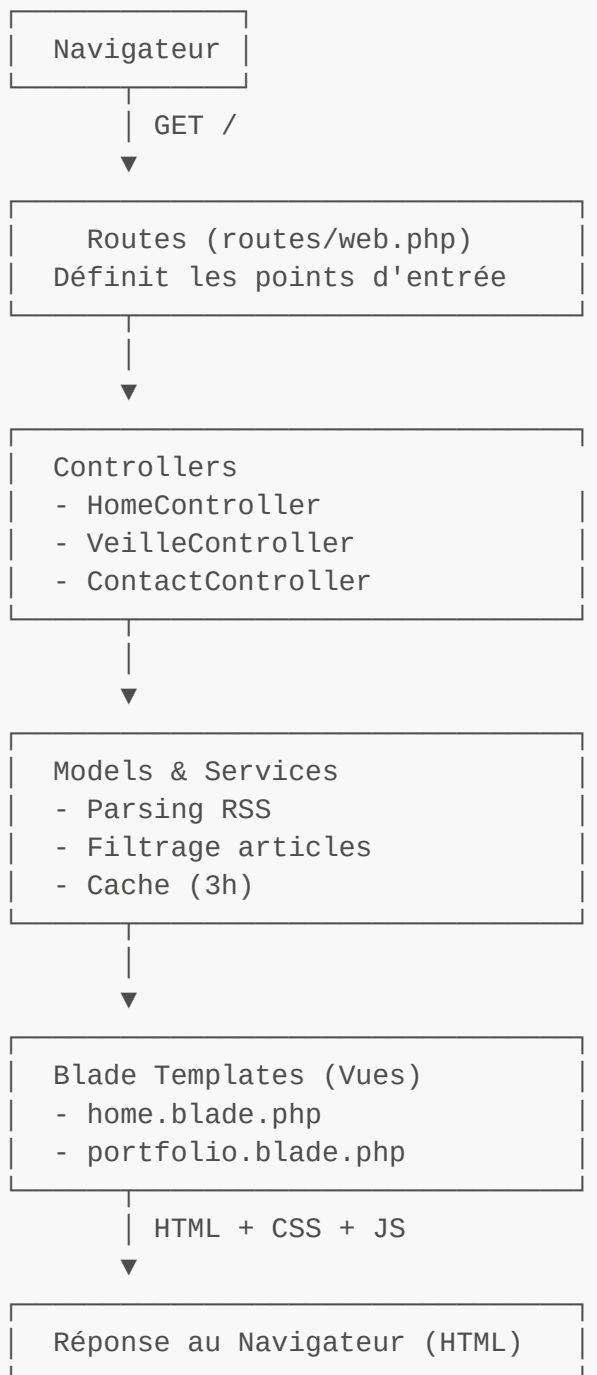
Le projet respecte strictement le pattern **Modèle-Vue-Contrôleur (MVC)** imposé par Laravel, favorisant la maintenance et la séparation des responsabilités.





- Diagramme Architecture MVC

- Localisation: Début de cette section
- Contenu: Schéma montrant Navigateur → Routes (web.php) → Contrôlers → Services/Models → Blade Views → Navigateur
- Dimensions recommandées: 1200x600px
- Légende: *Flux requête-réponse dans l'architecture MVC du portfolio*



3.2 Routing (routes/web.php)

```
// Page d'accueil - Affiche hero, carousel, veille, contact
Route::get('/', [HomeController::class, 'index'])->name('portfolio.home');

// Pages détail des projets
Route::get('/projects/{project}', function ($project) {
    return view("Projects.{ $project }");
})->name('project.{project}');

// API Veille - Récupère les articles filtrés (AJAX)
Route::get('/api/veille/articles', [VeilleController::class,
```

```
'getArticles']]);

// Formulaire de contact
Route::post('/contact', [ContactController::class, 'submit'])->
>name('portfolio.contact.submit');

// Téléchargement documentation
Route::get('/documents/documentation-portfolio.pdf', function () {
    return response()->file(public_path('documents/documentation-
portfolio.pdf'));
})->name('portfolio.documentation');
```

3.3 Les Contrôleurs Clés

HomeController

```
namespace App\Http\Controllers;

class HomeController extends Controller {
    public function index() {
        // Récupère les articles depuis VeilleController (ou cache)
        $articles = Cache::get('veille_articles') ?? [];

        return view('home', ['articles' => $articles]);
    }
}
```

Responsabilités:

- Charger la page d'accueil
- Injecter les données dynamiques (articles de veille)
- Gérer la logique simple d'affichage

VeilleController (Détailé en Section 5)

- Récupération des flux RSS
- Filtrage intelligent par sujet
- Gestion du cache (3h)
- Retour JSON pour AJAX

3.4 Les Vues (Blade Templates)

Utilisation du moteur de template **Blade** pour :

1. Héritage de layout :

```
@extends('layouts.app')    {{-- Héritage du layout principal --}}
@section('content')
```

```
{{-- Contenu spécifique de la page --}}
@endsection
```

2. Boucles et conditions :

```
@foreach($articles as $article)
    <article class="veille-card">
        <h3>{{ $article['title'] }}</h3>
        <p>{{ Str::limit($article['content'], 100) }}</p>
    </article>
@endforeach
```

3. Protection XSS (Injection de code malveillant) :

```
{{-- Échappe automatiquement les caractères HTML --}}
<p>{{ $user_input }}</p>  {{-- ✅ Sûr --}}

{{-- Si on veut vraiment du HTML, utiliser: --}}
<p>{!! html_entity_decode($trusted_html) !!}</p>  {{-- ⚠️ À faire avec soin --}}
```

4. SÉCURITÉ & QUALITÉ (BTS SIO)

4.1 Protection CSRF (Cross-Site Request Forgery)

Problème: Un attaquant pourrait créer un faux formulaire pour envoyer des emails depuis ton site.

Solution Laravel: Token CSRF obligatoire sur tous les formulaires POST

```
{{-- Chaque formulaire doit contenir ce token --}}
<form method="POST" action="{{ route('portfolio.contact.submit') }}">
    @csrf  {{-- Génère <input type="hidden" name="_token" value="..."> --}}
    <input name="email" type="email" required>
</form>
```

Le serveur vérifie que le token reçu correspond à celui de la session utilisateur.

Résultat: Si quelqu'un d'autre essaie de soumettre le formulaire depuis son site, la requête est rejetée (erreur 419).

4.2 Validation & Sanitization

Côté Backend (FormRequest)

```
namespace App\Http\Requests;

class ContactRequest extends FormRequest {
    public function rules() {
        return [
            'name' => 'required|string|max:100',
            'email' => 'required|email|max:255',
            'subject' => 'required|string|max:200',
            'message' => 'required|string|min:10|max:5000',
        ];
    }
}
```

Validation appliquée:

- **required** : Champ obligatoire
- **email** : Format email valide (utilise RFC 5322)
- **max:100** : Maximum 100 caractères
- **min:10** : Minimum 10 caractères

Si un champ ne respecte pas les règles → réponse HTTP 422 (Unprocessable Entity)

Côté Veille Technologique (Nettoyage RSS)

```
// Avant de stocker un article RSS, on nettoie le contenu HTML
$content = strip_tags($article['description']); // Supprime tous les tags HTML
$content = htmlspecialchars($content, ENT_QUOTES, 'UTF-8'); // Échappe les caractères spéciaux
```

Pourquoi? Un flux RSS malveillant (ou compromis) pourrait contenir du JavaScript. En supprimant les tags HTML, on neutralise ce risque.

4.3 Gestion des Erreurs AJAX

```
// Soumettre le formulaire en AJAX (sans rechargement)
document.getElementById('contactForm').addEventListener('submit', async (e) => {
    e.preventDefault();

    const formData = new FormData(e.target);

    try {
        const response = await fetch(e.target.action, {
            method: 'POST',
            body: formData
        });
    }
});
```

```
    if (!response.ok) {  
      // Erreur 422 = validation échouée  
      const errors = await response.json();  
      console.error('Erreurs:', errors);  
      // Afficher les erreurs sur chaque champ  
    } else {  
      // Succès = email envoyé  
      alert('Message envoyé avec succès!');  
    }  
  } catch (error) {  
    console.error('Erreur réseau:', error);  
  }  
});
```

Avantages:

- UX fluide (pas de rechargement de page)
- Affichage instant des erreurs
- Meilleure accessibilité

5. LE MOTEUR DE VEILLE TECHNOLOGIQUE

5.1 Problématique

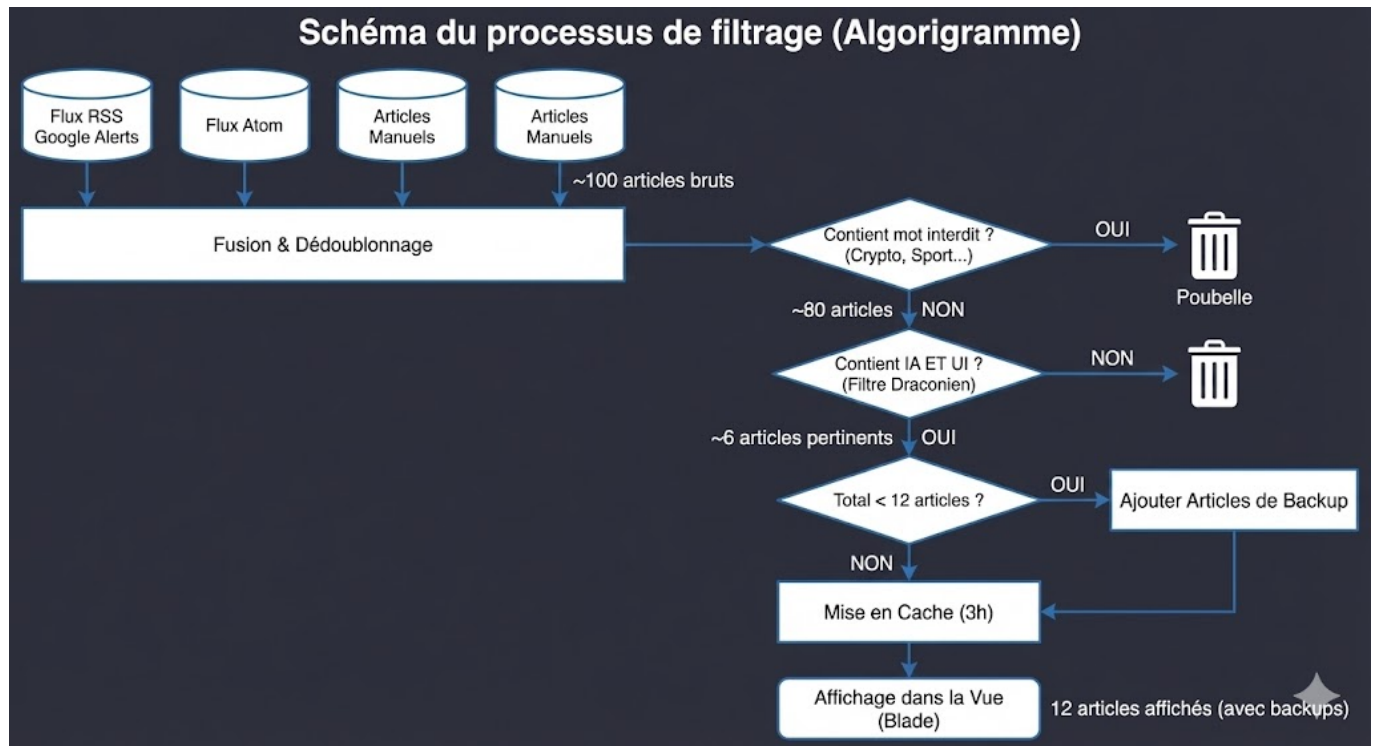
L'objectif était de suivre automatiquement mon sujet de veille : **"L'IA générative dans les interactions UI complexes"**.

Défi majeur: Les flux RSS bruts (Google Alerts) sont extrêmement "bruyants" :

```
× Articles hors-sujet récupérés:  
- "Équipe IA recrute à Paris" (c'est une offre d'emploi)  
- "Bitcoin atteint 50 000€" (crypto, hors-sujet)  
- "Nouvelle imprimante 3D IA" (pas du tout lié au frontend)  
- "Guide: Apprendre Python pour débutants" (trop basique)
```

Solution: Développer un algorithme de filtrage intelligent capable de ne garder que les articles **vraiment pertinents**.

5.2 L'Algorithme "Le Filet Dérivant"



- Schéma du processus de filtrage

- Localisation: Après la description du problème
- Contenu: Infographie montrant Flux RSS → Agrégation → Filtrage → Cache → Affichage
- Dimensions recommandées: 1400x700px
- Légende: *Flux de traitement des articles RSS: du brut au filtré*

Étape 1 : Agrégation Multi-Sources

Le **VeilleController** interroge **5 sources différentes** :

```

$feeds = [
    'https://www.google.com/alerts/feeds/.../11834831722007573275', // "AI
generate frontend code"
    'https://www.google.com/alerts/feeds/.../11613282588683397431', //
"code quality AI assistants"
    'https://www.google.com/alerts/feeds/.../12811262025504322800', //
"debugging AI code"
    'https://www.google.com/alerts/feeds/.../17806777925297812006', //
"GitHub Copilot for UI"
    'https://www.google.com/alerts/feeds/.../2894901103864049013', //
"low-code AI front-end"
];

// Pour chaque flux, faire une requête HTTP et parser le XML
foreach ($feeds as $feedUrl) {
    $articles = $this->parseFeed($feedUrl);
    $allArticles = array_merge($allArticles, $articles);
}
  
```

Parsing XML (SimpleXML + XPath):

```

private function parseFeed($feedUrl) {
    $response = Http::get($feedUrl);
    $xml = simplexml_load_string($response->body());

    $articles = [];

    // Compatible avec RSS <item> et Atom <entry>
    $entries = $xml->xpath('//item | //entry');

    foreach ($entries as $entry) {
        $articles[] = [
            'title' => (string)($entry->title ?? ''),
            'link'   => (string)($entry->link['href'] ?? $entry->link ??
            ''),
            'date'   => $this->formatDate((string)($entry->published ??
            $entry->pubDate ?? '')),
            'content' => strip_tags((string)($entry->description ?? $entry->
            >summary ?? '')),
            'image'  => $this->extractImage($entry),
            'category' => $this->extractCategory($entry),
        ];
    }

    return $articles;
}

```

Étape 2 : Ajout d'Articles Manuels Curatés

En plus des flux RSS, j'ajoute 6 articles **"Gold"** sélectionnés à la main :

```

$manualArticles = [
    [
        'title' => 'The Future of Code: From Syntax to AI-Guided Vibe
Engineering',
        'link'   => 'https://www.startuphub.ai/ai-news/ai-video/2025/...',
        'date'   => '14/12/2025',
        'content' => 'AI-generated code. Frontend development stands
at...',
        'category' => 'Frontend AI',
        'is_manual' => true,
    ],
    // ... 5 autres articles importants
];

// Fusionner dans la liste globale
$allArticles = array_merge($allArticles, $manualArticles);

```

Avantage: Garantir qu'on a toujours des articles de qualité, même si les flux RSS sont faibles ce jour-là.

Étape 3 : Déduplication Intelligente

```
$uniqueArticles = [];  
$seenTitles = [];  
  
foreach ($allArticles as $article) {  
    $titleKey = strtolower(trim($article['title']));  
  
    // Si on n'a pas encore vu ce titre (case-insensitive)  
    if (!isset($seenTitles[$titleKey])) {  
        $seenTitles[$titleKey] = true;  
        $uniqueArticles[] = $article;  
    }  
    // Sinon, on saute l'article (c'est un doublon)  
}  
}
```

Exemple:

Input:

- "AI for UI: The Future is Here" (de flux RSS 1)
- "ai for ui: the future is here" (même article, flux RSS 2)

Output:

- "AI for UI: The Future is Here" (gardé une seule fois)

Étape 4 : Le Cœur - Filtrage Draconien

C'est ici que la **vraie intelligence** se joue.

```
$relevantArticles = array_filter($uniqueArticles, function ($article) {  
    // Condition 1: Doit avoir une image  
    if (empty($article['image'])) {  
        return false; // Rejeté  
    }  
  
    $title = strtolower($article['title']);  
    $content = strtolower($article['content'] ?? '');  
    $combined = $title . ' ' . $content;  
  
    // Condition 2: Doit contenir un mot-clé IA obligatoire  
    $hasAIKeyword = (  
        strpos($combined, 'ai') !== false ||  
        strpos($combined, 'generative') !== false ||  
        strpos($combined, 'llm') !== false ||  
        strpos($combined, 'gpt') !== false ||  
        strpos($combined, 'copilot') !== false  
    );  
});
```

```
if (!$hasAIKeyword) {
    return false; // Rejeté (pas d'IA mentionné)
}

// Condition 3: Doit contenir un mot-clé Interface
$compositeKeywords = [
    'ui ai', 'ai ui', 'frontend ai', 'ai frontend',
    'interaction ai', 'ai interaction', 'ai-generated', 'ai generated'
];

$hasCompositeKeyword = false;
foreach ($compositeKeywords as $keyword) {
    if (strpos($combined, $keyword) !== false) {
        $hasCompositeKeyword = true;
        break;
    }
}

// Accepté si: composite keyword trouvé OU IA + contexte développeur
if ($hasCompositeKeyword) {
    return true; // ✅ Gardé
}

// Sinon, check contexte développeur
$hasDeveloperContext = (
    strpos($combined, 'developer') !== false ||
    strpos($combined, 'code') !== false ||
    strpos($combined, 'frontend') !== false ||
    strpos($combined, 'ux') !== false
);

return $hasCompositeKeyword || $hasAIKeyword && $hasDeveloperContext;
});
```

Table de décision:

Article	Contient IA?	Contient UI?	Composite?	Contexte Dev?	Résultat
"GitHub Copilot for Vue"	✅	✅	✅	-	✅ GARDÉ
"AI code generator trends"	✅	×	×	✅	✅ GARDÉ
"Learn Python with AI"	✅	×	×	×	×
"Bitcoin AI prediction"	✅	×	×	×	×
"Design systems & AI"	✅	✅	✅	-	✅ GARDÉ

Étape 5 : Tri par Relevance + Date

```

usort($relevantArticles, function ($a, $b) {
    $compositeKeywords = [
        'ui ai', 'frontend ai', 'interaction ai', 'ai-generated'
    ];

    $combined_a = strtolower($a['title'] . ' ' . ($a['content'] ?? ''));
    $combined_b = strtolower($b['title'] . ' ' . ($b['content'] ?? ''));

    $aHasComposite = false;
    $bHasComposite = false;

    foreach ($compositeKeywords as $kw) {
        if (strpos($combined_a, $kw) !== false) $aHasComposite = true;
        if (strpos($combined_b, $kw) !== false) $bHasComposite = true;
    }

    // Articles avec composite keywords en premier (plus pertinents)
    if ($aHasComposite && !$bHasComposite) return -1;
    if (!$aHasComposite && $bHasComposite) return 1;

    // Puis, par date décroissante (plus récent en premier)
    return strtotime($b['rawDate']) - strtotime($a['rawDate']);
});

```

Résultat du tri:

1. "AI-Generated UX: The Future" (Composite + Récent) ★★★
2. "Copilot Makes Frontend Faster" (Composite + Ancien) ★★
3. "AI and Design Systems" (Composite + Plus ancien) ★★
4. "AI Code Quality in 2025" (Non-composite + Récent) ★
5. "Learning AI for Developers" (Non-composite + Ancien) ★

Étape 6 : Retour des 6 Meilleurs + Caching

```

// Garder seulement les 6 meilleurs articles
$finalArticles = array_slice($relevantArticles, 0, 6);

// Les stocker en cache pour 3 heures (10 800 secondes)
Cache::put('veille_articles', $finalArticles, 10800);

return $finalArticles;

```

Pourquoi 3 heures?

- Pas trop court (sinon on spamme Google Alerts = bannissement IP)
- Pas trop long (les articles doivent être à jour)

- Optimal pour un usage personnel

5.3 Affichage Dynamique sur le Site

```
{{-- Dans home.blade.php --}}
<div class="veille-grid">
    @foreach($articles as $i => $article)
        <article class="veille-card">
            {{-- Catégorie avec fond coloré --}}
            <div class="veille-image">
                <span style="color: var(--primary); font-weight: bold;">
                    {{ $article['category'] }}
                </span>
            </div>

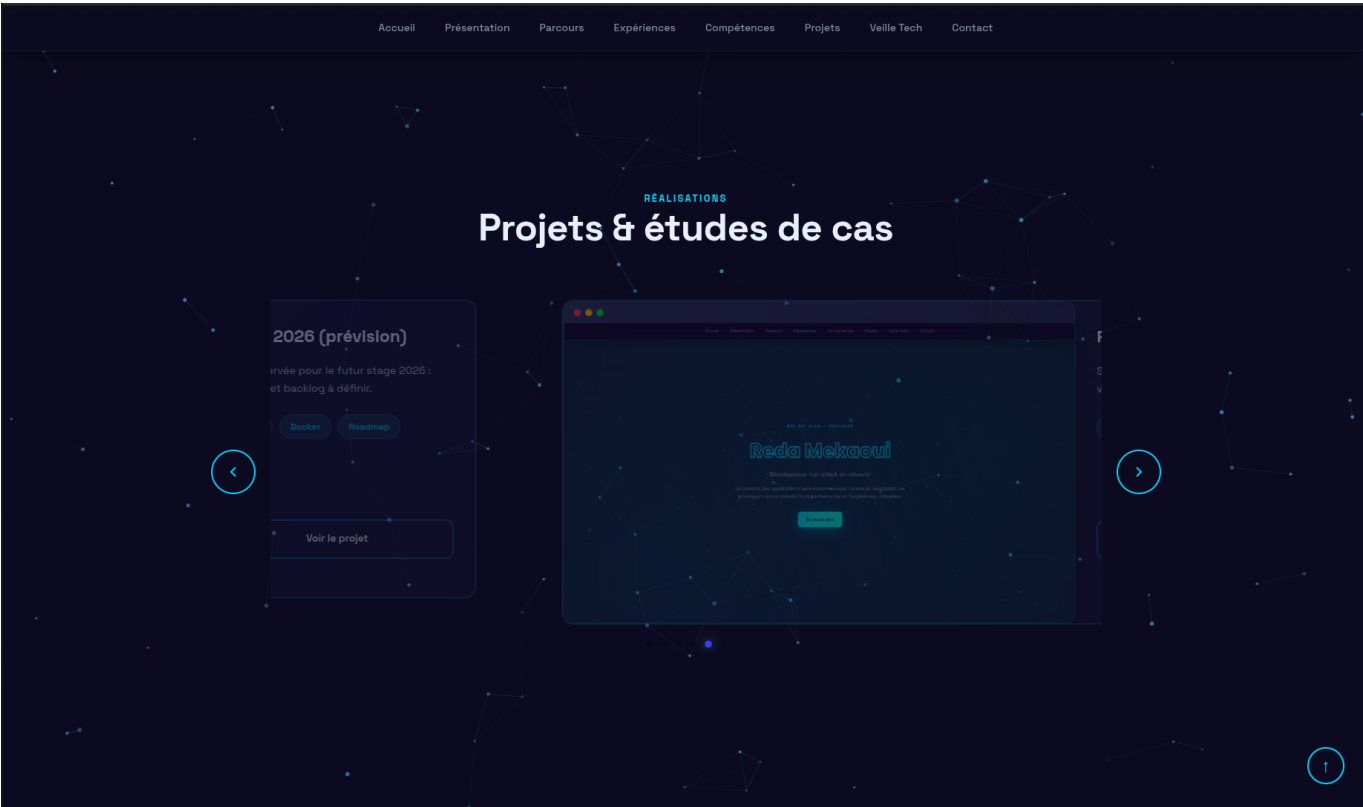
            {{-- Titre et date --}}
            <p class="veille-title">
                {{ Str::limit($article['title'], 60) }}
            </p>
            <p class="veille-date">
                Publié le : {{ $article['date'] }}
            </p>

            {{-- Lien vers l'article original --}}
            <a class="veille-button" href="{{ $article['link'] }}"
target="_blank">
                Lire l'article
            </a>
        </article>
    @endforeach
</div>
```

5.4 Points Clés de l'Implémentation

- ✓ **Robustesse:** Gestion des erreurs XML, fallback sur images par défaut
- ✓ **Performance:** Cache 3h pour éviter les appels inutiles
- ✓ **Relevance:** Tri composite keywords + date
- ✓ **Maintenabilité:** Code lisible, commentés, logique claire
- ✓ **Sécurité:** Strip tags, htmlspecialchars, validation des URLs

6. INTERFACE UTILISATEUR



- Section Projets avec effet Glassmorphism

- Localisation: Au début de cette section
- Contenu: Grille des cartes projets avec l'effet de flou arrière-plan, hover effects
- Dimensions recommandées: 1400x500px
- Légende: *Cartes projets avec design Glassmorphism et animations au survol*

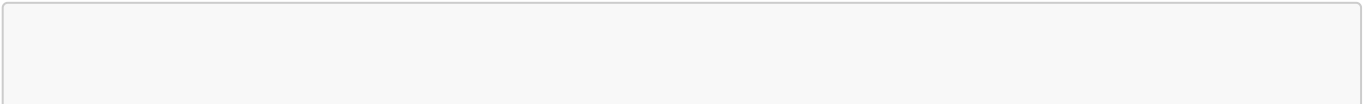
6.1 Design System & Thème

```
/* Variables CSS pour une maintenance facile */
:root {
  --primary: #00d9ff;      /* Cyan néon */
  --accent: #00ffcc;       /* Turquoise */
  --bg-dark: #0f1419;      /* Fond très sombre */
  --text: #ffffff;          /* Texte blanc */
  --text-secondary: #cbd5e1;
  --muted: #94a3b8;
}
```

Typographie:

- Titres: Poppins Bold 700
- Corps: Inter Regular 400, 14-16px
- Code: JetBrains Mono (monospace)

6.2 Glassmorphism Effect



```
.project-card {  
  background: rgba(20, 25, 40, 0.6); /* Fond semi-transparent */  
  backdrop-filter: blur(12px);      /* Flou arrière-plan */  
  border: 1px solid rgba(0, 217, 255, 0.1);  
  border-radius: 20px;  
}  
  
.project-card:hover {  
  border-color: var(--primary);  
  box-shadow: 0 20px 40px -10px rgba(0, 217, 255, 0.15);  
  transform: translateY(-10px);  
}
```

Avantage: Rendu moderne, léger, sophistiqué.

6.3 Animations

Swiper.js - Carrousel Tactile

```
import Swiper from 'swiper';  
import { Navigation, Pagination } from 'swiper/modules';  
  
new Swiper('.projects-swiper', {  
  modules: [Navigation, Pagination],  
  slidesPerView: 1,  
  spaceBetween: 30,  
  loop: true,  
  grabCursor: true, // Curseur "main" au hover  
  navigation: {  
    nextEl: '.carousel-next',  
    prevEl: '.carousel-prev',  
  },  
  pagination: {  
    el: '.swiper-pagination',  
    clickable: true,  
  },  
});
```

Anime.js - Animation SVG Fluide

```
import anime from 'animejs';  
  
anime({  
  targets: '.hero-title .line',  
  strokeDashoffset: [length, 0],  
  duration: 2500,  
  easing: 'easeInOutQuad',  
  loop: true,  
});
```

```

    direction: 'alternate'
  });

```

Canvas API - Particules en Arrière-Plan

```

// Crée 120 particules avec mouvement, connexions, opacité
class Particle {
  constructor() {
    this.x = Math.random() * canvas.width;
    this.y = Math.random() * canvas.height;
    this.speedX = (Math.random() - 0.5) * 0.6;
    this.speedY = (Math.random() - 0.5) * 0.6;
  }

  draw() {
    ctx.fillStyle = 'rgba(99, 209, 255, 0.6)';
    ctx.arc(this.x, this.y, this.size, 0, Math.PI * 2);
    ctx.fill();
  }
}

const animate = () => {
  particles.forEach(p => { p.update(); p.draw(); });
  connect(); // Dessine les lignes entre particules proches
  requestAnimationFrame(animate);
};

```

6.4 Responsive Design

```

/* Mobile first */
.veille-grid {
  display: grid;
  grid-template-columns: 1fr; /* 1 colonne sur mobile */
  gap: 20px;
}

/* Tablette */
@media (min-width: 768px) {
  .veille-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

/* Desktop */
@media (min-width: 1200px) {
  .veille-grid {
    grid-template-columns: repeat(3, 1fr); /* 3 colonnes */
  }
}

```

CONCLUSION

Ce projet démontre la capacité à :

- ✓ **Architecturer** une application Laravel structurée (MVC)
- ✓ **Développer** un algorithme de filtrage intelligent (veille technologique)
- ✓ **Sécuriser** une application web (CSRF, validation, sanitization)
- ✓ **Conteneuriser** un environnement de développement (Docker + Sail)
- ✓ **Animer** une interface fluide et moderne (Canvas, Anime, Swiper)

Technologies démontrées:

- Backend: Laravel 11, PHP 8.2, Blade Templates, Composer
- Frontend: JavaScript ES6+, CSS3, Vite
- DevOps: Docker, Docker Compose, Laravel Sail
- Data: RSS/Atom Parsing, XML, XPath

Compétences BTS SIO validées:

- Architecture logicielle (MVC)
- Sécurité web (CSRF, validation, sanitization)
- Intégration APIs externes (Google Alerts)
- Optimisation performance (cache, bundling)